

Data Structure Lab4 : Singly Linked List 2022-2023

Topics

1. Implement Node Class
2. Generics
3. Implement SinglyLinkedList Class
4. Implement Basic Methods of SinglyLinkedList
 - isEmpty()
 - size()
 - first()
 - last()
 - addFirst()
 - addLast()
 - removeFirst()

Homework

1. develop an implementation of the equals method in the context of the SinglyLinkedList class.

```
public class Lab4<T> {  
    private static class Node<T> {  
        T data;  
        Node<T> next;  
  
        public Node(T d, Node<T> n) {  
            data = d;  
            next = n;  
        }  
    }  
}  
  
private Node<T> start = null;  
private Node<T> end = null;  
private int count = 0;  
  
public Lab4() {}  
  
public int getCount() {  
    return count;  
}  
  
public boolean isVacant() {  
    return count == 0;  
}
```

Data Structure Lab4 : Singly Linked List 2022-2023

```
public void insertLast(T d) {
    Node<T> newNode = new Node<>(d, null);
    if (isVacant()) {
        start = newNode;
    } else {
        end.next = newNode;
    }
    end = newNode;
    count++;
}

@Override
public boolean sameAs(Object obj) {
    if (this == obj) return true; // نفس الكائن
    if (!(obj instanceof Lab4<?>)) return false;

    Lab4<?> other = (Lab4<?>) obj;
    if (this.count != other.count) return false; // تأكد من أن الحجم متساوٍ

    Node<T> currA = this.start;
    Node<?> currB = other.start;

    while (currA != null) {
        if (!currA.data.equals(currB.data)) return false; // القيم غير متساوية
        currA = currA.next;
        currB = currB.next;
    }
    return true; // القائمتان متطابقتان
}

public static void main(String[] args) {
    Lab4<Integer> list1 = new Lab4<>();
    Lab4<Integer> list2 = new Lab4<>();

    list1.insertLast(1);
    list1.insertLast(2);
    list1.insertLast(3);

    list2.insertLast(1);
    list2.insertLast(2);
    list2.insertLast(3);

    System.out.println("هل القائمتان متشابهتان؟" + list1.sameAs(list2)); true
}
}
```

2. SinglyLinkedList في سياق فئة equals قم بتطوير وتنفيذ طريقة

```
public class Lab4<T> {

    private static class Node<T> {

        T data;
```

Data Structure Lab4 : Singly Linked List 2022-2023

```
Node<T> next;
```

```
public Node(T d, Node<T> n) {  
    data = d;  
    next = n;  
}  
}
```

```
private Node<T> start = null;  
private Node<T> end = null;  
private int count = 0;
```

// دالة إضافة عنصر في النهاية

```
public void insertAtEnd(T data) {  
    Node<T> newNode = new Node<>(data, null);  
    if (isEmpty()) {  
        start = newNode;  
    } else {  
        end.next = newNode;  
    }  
    end = newNode;  
    count++;  
}
```

Data Structure Lab4 : Singly Linked List 2022-2023

```
}
```

```
// دالة التحقق إذا كانت القائمة فارغة
```

```
public boolean isEmpty() {
```

```
    return count == 0;
```

```
}
```

```
// دالة استرجاع عدد العناصر
```

```
public int getSize() {
```

```
    return count;
```

```
}
```

```
// للتحقق من إذا كانت القائمتان متساويتين equals دالة
```

```
@Override
```

```
public boolean areListsEqual(Object obj) {
```

```
    if (this == obj) return true; // نفس الكائن
```

```
    if (!(obj instanceof Lab4<?>)) return false;
```

```
    Lab4<?> other = (Lab4<?>) obj;
```

```
    if (this.count != other.count) return false; // الحجم مختلف
```

```
    Node<T> currentA = this.start;
```

Data Structure Lab4 : Singly Linked List 2022-2023

```
Node<> currentB = other.start;
```

```
while (currentA != null) {
```

```
    if (!currentA.data.equals(currentB.data)) return false; // القيم غير متساوية
```

```
    currentA = currentA.next;
```

```
    currentB = currentB.next;
```

```
}
```

```
return true; // القائمتان متساويتان
```

```
}
```

```
// دالة لاختبار الكود
```

```
public static void main(String[] args) {
```

```
    Lab4<Integer> list1 = new Lab4<>();
```

```
    Lab4<Integer> list2 = new Lab4<>();
```

```
    // إضافة بعض العناصر للقائمتين
```

```
    list1.insertAtEnd(1);
```

```
    list1.insertAtEnd(2);
```

```
    list1.insertAtEnd(3);
```

```
    list2.insertAtEnd(1);
```

Data Structure Lab4 : Singly Linked List 2022-2023

```
list2.insertAtEnd(2);
```

```
list2.insertAtEnd(3);
```

```
// اختبار المساواة بين القائمتين
```

```
System.out.println("هل القائمتان متساويتان؟" + list1.areListsEqual(list2)); true
```

```
}
```

```
}
```

3. Give an algorithm for finding the second-to-last node in a singly linked list in which the last node is indicated by a null next reference.

```
public Node<T> secondToLastNode() {  
    if (head == null || head.next == null) {  
        return null; // القائمة فارغة أو تحتوي على عنصر واحد فقط  
    }  
}
```

```
Node<T> current = head;  
while (current.next != null && current.next.next != null) {  
    current = current.next;  
}  
return current; // العنصر قبل الأخير  
}
```

4. Give an implementation of the size() method for the SinglyLinkedList class, assuming that we did not maintain size as an instance variable.

```
public int size() {  
    int count = 0;  
    Node<T> current = head;  
    while (current != null) {
```

Data Structure Lab4 : Singly Linked List 2022-2023

```
        count++;
        current = current.next;
    }
    return count;
}
```

5. Implement a rotate() method in the SinglyLinkedList class, which has semantics equal to addLast(removeFirst()), yet without creating any new node.

```
public void rotate() {
    if (head == null || head.next == null) {
        return; // القائمة فارغة أو تحتوي على عنصر واحد فقط
    }

    Node<T> first = head;
    head = head.next;
    Node<T> current = head;
    while (current.next != null) {
        current = current.next;
    }
    current.next = first;
    first.next = null;
}
```

6. Describe an algorithm for concatenating two singly linked lists L and M, into a single list L' that contains all the nodes of L followed by all the nodes of M.

```
public void concatenate(SinglyLinkedList<T> listM) {
    if (head == null) {
        head = listM.head;
        return;
    }
}
```

Data Structure Lab4 : Singly Linked List 2022-2023

```
Node<T> current = head;
while (current.next != null) {
    current = current.next;
}
current.next = listM.head;
}
```

7. Describe in detail an algorithm for reversing a singly linked list L using only a constant amount of additional space.

```
public void reverse() {
    Node<T> prev = null;
    Node<T> current = head;
    Node<T> next = null;

    while (current != null) {
        next = current.next;
        current.next = prev;
        prev = current;
        current = next;
    }

    head = prev;
}
```